

Aplicaciones Web - Curso 2025/2026

Convocatoria ordinaria (14/01/2026) - Duración: **3 horas**

Puntuación máxima del examen: **10 puntos**

Puntuación mínima en el examen para poder aprobar la asignatura: **5 puntos**

INSTRUCCIONES

Descarga y descomprime el archivo proporcionado para el examen en un directorio con permisos de escritura. El proyecto incluye todos los recursos necesarios para su desarrollo: estructura del servidor, vistas, recursos estáticos, dependencias y ficheros de configuración.

Antes de comenzar, abre el proyecto con **Visual Studio Code** y comprueba que la estructura es correcta y que el servidor arranca sin errores. El proyecto ya está configurado y funcional; **no es necesario generar la estructura ni ejecutar express-generator**.

Durante el examen se trabajará **con datos en memoria**. En la resolución de los ejercicios solo podrán utilizarse los módulos instalados en la carpeta `node_modules` proporcionada.

Los recursos estáticos se encuentran en la carpeta `public`. Se permite añadir nuevos estilos en `styles.css` y/o crear nuevos ficheros JavaScript propios, siempre que estén correctamente enlazados.

El servidor deberá ejecutarse en el puerto **3000**.

Los ejercicios entregados deben funcionar correctamente. No se admitirán errores de recursos no encontrados.

El diseño no es el aspecto principal del examen, pero se exige que los resultados se muestren de forma **clara, ordenada y comprensible**, valorándose positivamente una presentación cuidada y penalizándose soluciones descuidadas o poco legibles.

INSTRUCCIONES DE ENTREGA

Para entregar la solución del examen, crea un fichero **NombreApellidos.zip** (usar sólo ZIP). En él debes incluir todos los ejercicios bien en carpetas separadas o todas juntas siempre **exceptuando** la carpeta **node-modules** y los ficheros **package.json** y **package-lock.json**. Haz doble clic en el icono del escritorio denominado: **“EXAMENES en LABs entregas”**, y dentro de la ventana que aparece, haz doble clic en **“ALUMNOS entrega de prácticas y exámenes”**. Se abre otra ventana en la que debes seleccionar el archivo **zip** en el panel inferior izquierdo y arrastrarlo al panel inferior derecho (o utiliza el botón derecho del ratón y la opción **Subir**), como se indica en la figura.

Antes de abandonar el laboratorio debes pasar por el puesto del profesor para asegurarte de que lo que se ve en el puesto del profesor es lo que has entregado (**comprobando el tamaño del archivo**) y **FIRMAR EN LA HOJA DE ENTREGAS**.

Pregunta 1 [3 puntos]

La Biblioteca Central de la UCM desea disponer de una página que permita consultar y filtrar la información de su personal auxiliar, a partir de los datos iniciales proporcionados en el fichero `personal.json`.

Al arrancar el servidor, se deberán cargar los datos del fichero `personal.json`, ubicado en la carpeta **data**, y almacenarlos en una estructura en memoria. Esta carga deberá realizarse **una sola vez**.

Desarrolla una página **accesible** que muestre un listado del personal auxiliar en la ruta **`http://localhost:3000/listado`**, indicando al menos:

- Nombre completo
- Correo electrónico
- Turno
- Imagen asociada

Las imágenes se encuentran en la carpeta `/public/imagenes`.

En aquellos casos en los que el campo correspondiente a la imagen esté vacío, se deberá mostrar la imagen por defecto `noUser.png`.

En la página de listado se deberán implementar los siguientes filtros, utilizando los mecanismos vistos en la asignatura y haciendo uso de **`underscore.js`** para el filtrado de los datos.

1. Filtro por turno: Implementar un filtro por turno (mañana / tarde) mediante un desplegable. Este filtro se desarrollará utilizando **`query string`**.
2. Filtro por identificador: Implementar un filtro por identificador numérico, mediante un desplegable que permita seleccionar una persona concreta. La consulta se realizará utilizando una **URL paramétrica**.
3. Filtro por nombre: Implementar un filtro por nombre mediante una caja de texto, de forma que el criterio de búsqueda permita coincidencias parciales (búsqueda por subcadena) y no sea sensible a mayúsculas y minúsculas, sin necesidad de introducir el nombre completo. El criterio de búsqueda se enviará mediante una **petición POST** y los datos se obtendrán a partir del cuerpo de la petición.

Pregunta 2 [4 puntos]

Partiendo de la página de listado y filtros desarrollada en la pregunta anterior, se deberá ampliar la aplicación para permitir la gestión completa del personal auxiliar, implementando las operaciones de alta, modificación y eliminación lógica sobre los datos gestionados en memoria.

1. **Alta de personal:** Implementa un formulario accesible en la ruta **`http://localhost:3000/alta`** que permita dar de alta nuevo personal auxiliar, solicitando los siguientes datos:
 - a. Nombre completo (Nombre + Apellidos)
 - b. Correo electrónico
 - c. Contraseña
 - d. Turno (mañana / tarde)
 - e. Imagen asociada

El formulario deberá enviarse mediante una petición POST.

La validación de los datos deberá realizarse utilizando `express-validator`, haciendo uso obligatorio de `validationResult(req).mapped()`

Se deberán cumplir las siguientes condiciones:

- Ningún campo podrá estar vacío, excepto la imagen.
- El correo electrónico deberá pertenecer al dominio `@ucm.es`.
- La contraseña deberá tener una longitud mínima de 6 caracteres y máxima de 10 e incluir al menos una letra mayúscula, una letra minúscula, un dígito y un carácter especial.
- La imagen asociada se gestionará mediante `multer`, utilizando `single()`. La imagen se almacenará en la carpeta `/public/imagenes`. Si no se selecciona ninguna imagen, el campo correspondiente quedará como cadena vacía.
- Si los datos introducidos son correctos, el nuevo registro se añadirá a la estructura en memoria, asignándole un identificador numérico consecutivo y el atributo activo con valor 1.

2. **Modificación de datos del personal:** Desde el listado se deberá permitir la modificación de los datos de una persona concreta accediendo a la ruta **`http://localhost:3000/editar/:id`**.
 - El acceso a la edición se realizará mediante una URL paramétrica, utilizando el identificador numérico.
 - El formulario de edición deberá mostrarse precargado con los datos actuales.
 - Se deberán aplicar las mismas validaciones que en el alta (excepto la contraseña, que podrá mantenerse sin cambios si no se modifica).
 - En la operación de modificación se permitirá seleccionar una imagen y subirla al servidor utilizando `multer`, desde la carpeta `/imagenes_cliente`, almacenándose posteriormente a esta operación de modificación en la carpeta `/public/imagenes`. Esta operación deberá realizarse al menos para las **dos últimas personas almacenadas**, cuyo valor del campo `imagen` es una cadena vacía (`""`).
 - En esta operación se deberá permitir establecer el estado activo / inactivo de la persona.
3. **Eliminación lógica de una persona:** Se deberá implementar la eliminación lógica de una persona concreta, accesible desde el listado mediante la ruta: **`http://localhost:3000/eliminar/:id`**.
 - La eliminación lógica consistirá en modificar el valor del atributo activo a 0, sin eliminar físicamente el registro de la estructura en memoria.
 - A partir de este momento, en la página de listado desarrollada en la **pregunta anterior** solo deberán mostrarse aquellos registros cuyo atributo activo tenga valor 1.

En este ejercicio es obligatoria la gestión de los errores 404 y 500 mediante la implementación de middlewares. Se valorará muy positivamente la refactorización del código así como la visualización de los resultados.

Pregunta 3 [3 puntos]

La Biblioteca Central de la UCM desea mejorar la accesibilidad y usabilidad de la aplicación, permitiendo al usuario adaptar determinados aspectos visuales de la interfaz, así como obtener información del personal auxiliar mediante un **servicio web** consumido de forma asíncrona desde el cliente.

Implementa un **servicio web** accesible en la ruta **`http://localhost:3000/api/personal`** que devuelva, en formato JSON, la información del personal auxiliar activo, incluyendo al menos el identificador, nombre completo, correo y turno. El servicio web no deberá devolver ninguna vista, únicamente datos en formato JSON

Asimismo, se deberá desarrollar una nueva página independiente, accesible mediante la ruta:

<http://localhost:3000/accesibilidad>

En la página /accesibilidad se deberán implementar opciones de accesibilidad visual que permitan al usuario modificar dinámicamente el color de fondo, el color del texto y el tamaño del texto. Los cambios deberán aplicarse de forma inmediata sobre la propia página, mejorando la legibilidad y usabilidad de la aplicación.

Las opciones de accesibilidad seleccionadas deberán almacenarse mediante **cookies**, de forma que, al recargar la página o acceder nuevamente a dicha página, se recuperen automáticamente las preferencias guardadas y se apliquen de nuevo los estilos seleccionados. La lectura y escritura de las cookies deberá realizarse desde el servidor.

Las cookies utilizadas en esta pregunta se emplearán exclusivamente para almacenar las preferencias de accesibilidad seleccionadas en la página /accesibilidad. Dichas cookies deberán leerse únicamente al acceder a esta página, aplicándose los estilos correspondientes si existen valores almacenados.